

(https://www.domotique-store.fr/)

Accueil (http://tutoriels.domotique-store.fr/index.html)

Toutes les catégories (http://tutoriels.domotique-store.fr/showcat.html)

Choisir / Concevoir son installation domotique (/category/1/choisir-_-concevoir-son-installation-domotique.html)

Technologies Domotiques (/category/3/technologies-domotiques.html)

Box domotiques (/showcat.html)

- Zipabox de Zipato (/category/4/zipabox-de-zipato.html)
- Home Center de Fibaro (/category/7/home-center-de-fibaro.html)
  - Home Center 2 (/category/5/home-center-2.html)
    - Tutoriels (/category/76/tutoriels.html)
  - Home Center Lite (/category/75/home-center-lite.html)
- Vera (VeraEdge, VeraLite, Vera3) (/category/6/vera-veraedge-veralite-vera3.html)

eedomus (/category/24/eedomus.html)

Zibase de Zodianet (/category/25/zibase-de-zodianet.html)

BeNext iHome (/category/85/benext-ihome.html)

Modules domotiques Z-wave (/category/14/modules-domotiques-z_wave.html)

Modules domotiques RF433 (/category/16/modules-domotiques-rf433.html)

Base de données d'API (/category/17/base-de-donnees-d_api.html)

Caméras IP (/category/71/cameras-ip.html)

Box domotiques (/category/7/box-domotiques.html) » Home Center de Fibaro (/category/5/home-center-de-fibaro.html) » Home Center 2 (/category/74/home-center-2.html) » Tutoriels (/category/76/tutoriels.html)

G+1 +2 Recommander ce contenu sur Google
Tweeter
J'aime 9 Recommander Partager 0

Niveau expert

Introduction à la programmation en LUA sur la box domotique (http://Pack-Fibaro-HC2-Home-Center-6-micro-modules.domotique-store.fr)Fibaro (http://fibaro.domotique-store.fr) Home Center 2 (http://Pack-Fibaro-HC2-Home-Center-6-micro-modules.domotique-store.fr)

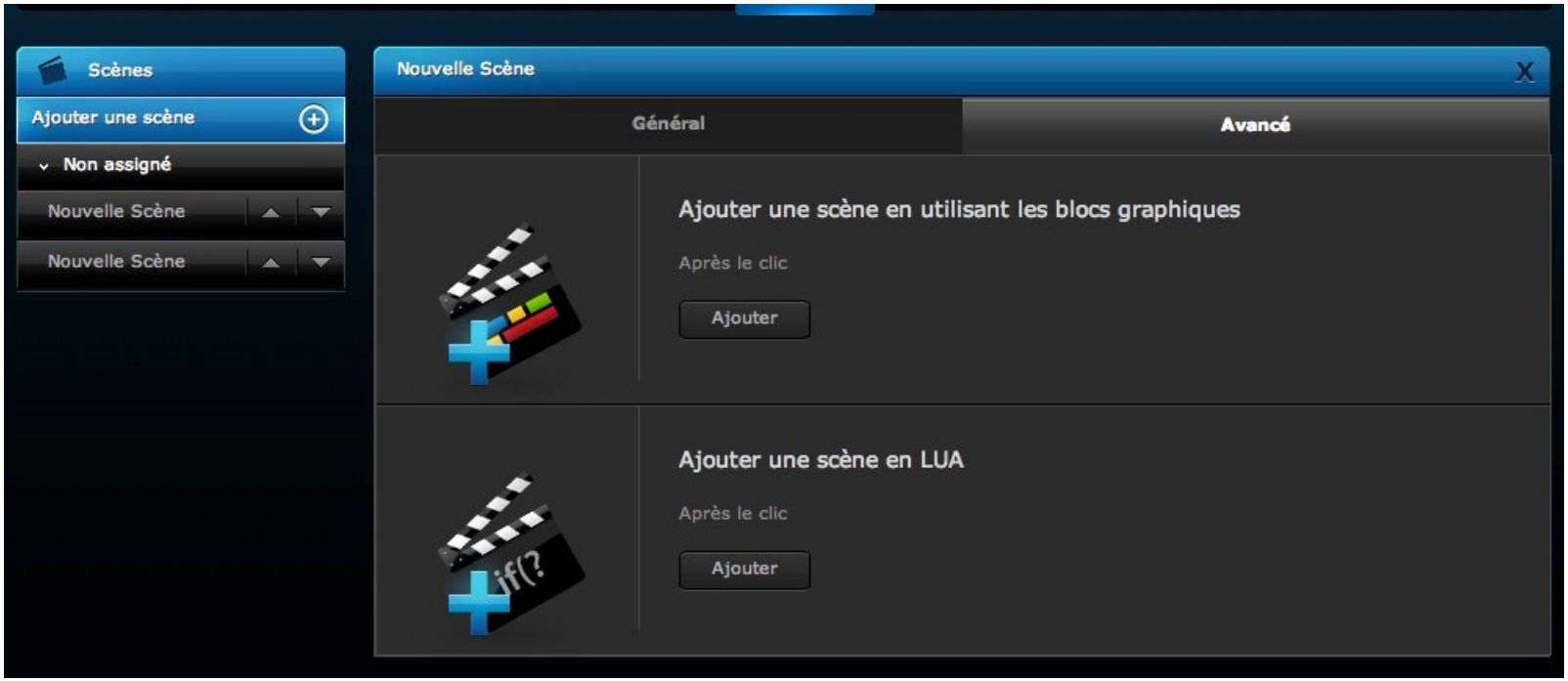
Dans cette introduction, vous apprendrez les bases de la programmation de scènes via le langage Lua.

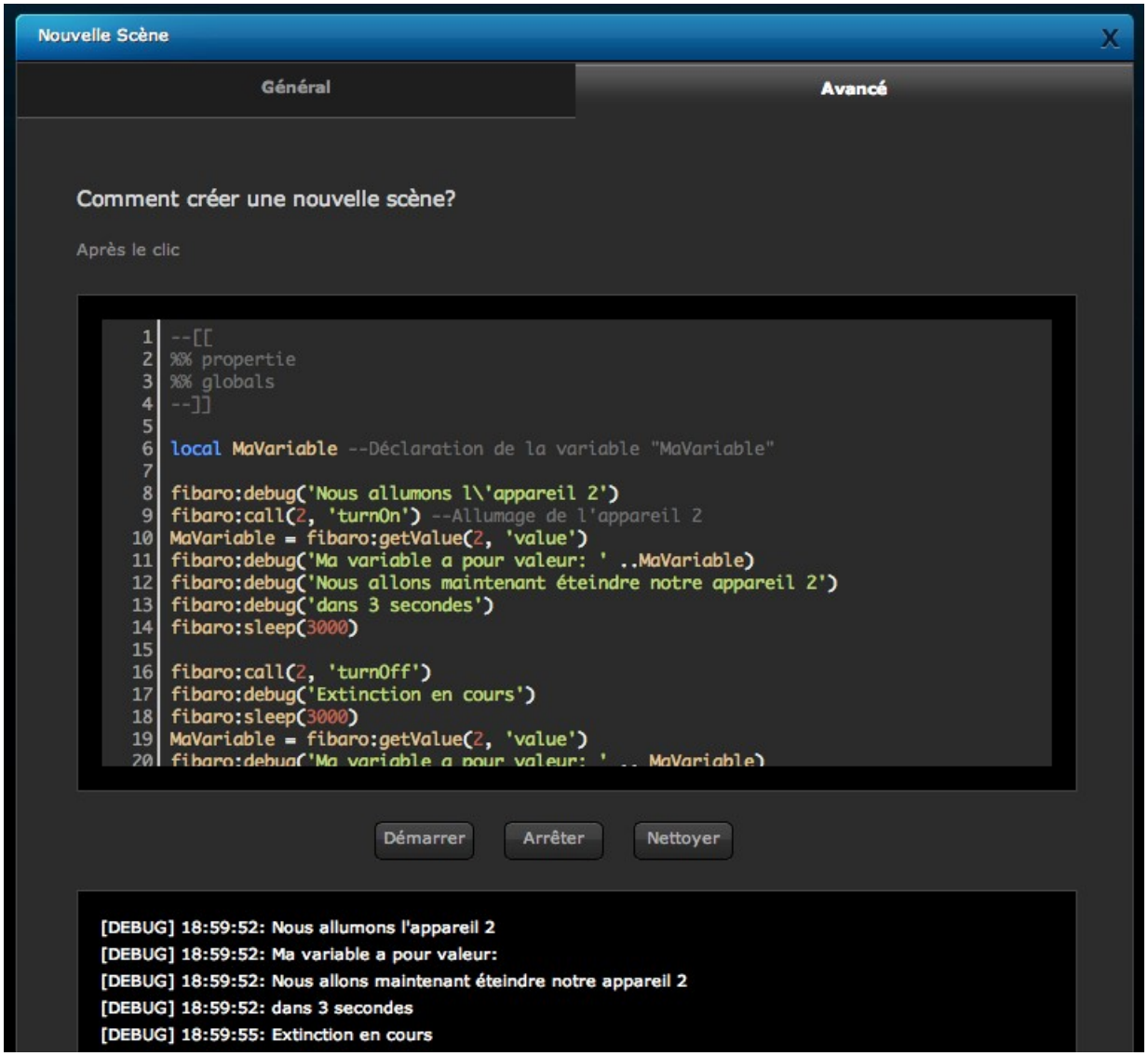
AJOUT DE SCÈNE EN LUA

La nouvelle API permet de programmer des scènes en langage de programmation de scripts LUA.

Lors de la création d'une nouvelle scène, vous pouvez choisir si la scène sera mis en place dans les blocs ou via un script LUA.




Scènes
Ajouter une scène
Non assigné
Nouvelle Scène
Nouvelle Scène
Nouvelle Scène
Général
Avancé
Ajouter une scène en utilisant les blocs graphiques
Après le clic
Ajouter
Ajouter une scène en LUA
Après le clic
Ajouter



Saisir le script LUA dans la fenêtre d'édition de code.

LUA fonctionne comme d'autres langages de programmation avec des lignes les unes au dessus des autres.
Si vous connaissez déjà d'autres langages de programmation, il n'est pas ici nécessaire de finir les lignes par un symbole (vous pouvez tout de même finir par une virgule mais LUA l'ignorera).

Chaque scène écrite en LUA doit toujours commencer avec cette entête :

```
1 --[[
2 %% properties
3 %% globals
4 -]]
```

L'en-tête est absolument indispensable pour n'importe quelle scène.

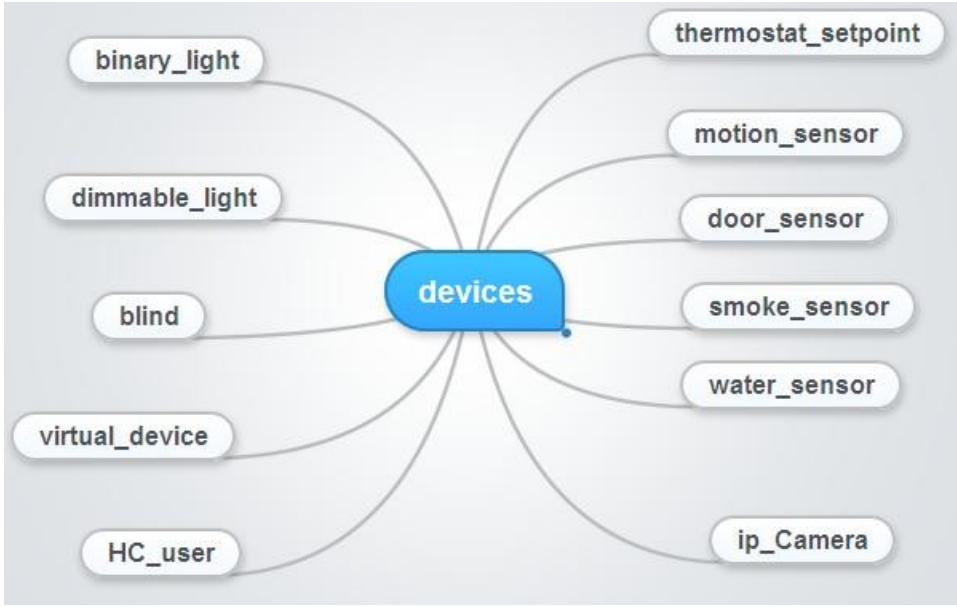
Sous le titre "%% properties" sont définis les "trigger" / "déclencheurs" et sous le titre "%% globals" sont définies les variables globales

Tout ceci sera décrit plus en détail un peu plus loin.

LE MATÉRIEL

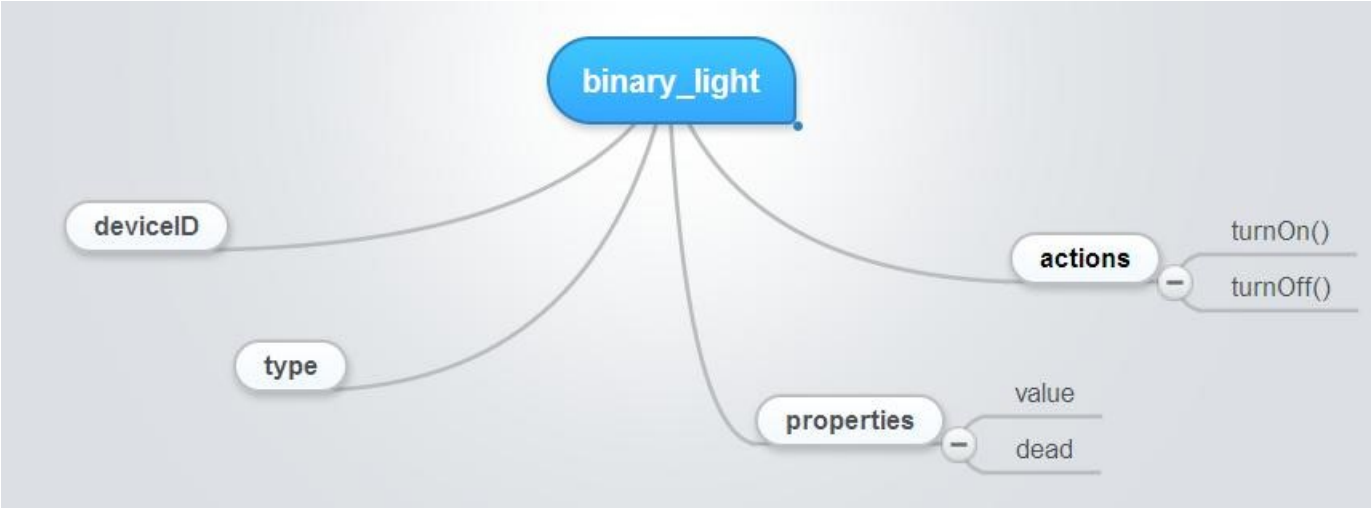
Vous devez d'abord comprendre le fonctionnement de l'API pour les différents types d'équipements.

Voici les principaux :



Les modules / appareils ajoutés au panneau "modules" du Home Center 2 sont indiqués avec type spécifique :

- binary_light – Simple relais ON/OFF par exemple.
- dimmable_light – Variateur (dimmer) ou certaines sirènes Fortrezz par exemple.
- blind – Volets roulants, rideaux motorisés, etc.
- virtual_device – Dispositif virtuels préalablement créés par l'utilisateur.
- thermostat_setpoint – Têtes thermostatiques (Danfoss Linving Connect par exemple).
- motion_sensor – Détecteurs de mouvements.
- door_sensor – Détecteur d'ouverture (porte, fenêtre, etc.)
- smoke_sensor – Détecteur de fumée.
- water_sensor – Détecteur d'inondation.
- ip_Camera – Caméra IP.



Avec la nouvelle API, les périphériques du système sont caractérisés par:

deviceID = Un numéro unique attribué par le HC2 et définissant le module,
Type = Le type de périphérique (ex: binary_light)
properties = Ses propriétés (ex: valeur, lien mort) : Donne l'état de l'appareil (détection de mouvement, interrupteur allumé, volet ouvert, etc). Ces valeurs sont en lecture seule. (détecteur de mouvement est compromise, l'interrupteur marche / arrêt est allumé et que l'obturateur est ouvert)
actions = actions (turnON(), turnOFF(), setValue()) : des actions qui peuvent être effectuées sur l'appareil (allumer : turnON(), éteindre turnOFF(), fixer un niveau d'allumage de la lampe setValue()).

La liste des API par appareils est disponible ici (http://docs.fibaro.com/?page_id=1090) (articles en Polonais sur le site de Fibaro, la traduction sera proposée ultérieurement)

NOTRE PREMIER SCRIPT EN LUA (fonction "call")

Nous allons commencer par une action simple : commander un module de type commutateur ON/OFF.

Pour ce faire, nous allons utiliser une des fonctions les plus élémentaires du HC2 : "call" (demander).

Vous trouverez plus d'informations sur la commande "Call" ici (http://docs.fibaro.com/?page_id=885) (article en Polonais)

Cette action s'écrit comme ceci : fibaro:call(deviceID, actionName, …)

Elle envoie une demande d'action à un équipement / module.

Elle prends au moins deux arguments :

- deviceID – Le numéro identifiant du module
- actionName - Le nom de l'action (ex: TurnOn, TurnOff, SetValue) - Les actions possibles dépendent des appareils et sont listées ici (http://docs.fibaro.com/?page_id=1090):

Exemple avec un micro-module commutateur (exemple: FGS211) dont l'ID serait "2" :

```
1 --[[
2 %% properties
```

```
3 %% globals
4 --]]
5
6 fibaro:call(2, 'turnOn' ) -- nous allumons le module dont l'identifiant est 2
```

La première partie du script est l'entête standard (ligne 1 à 4), la deuxième partie contient les actions (ligne 6)

```
1 --vous pouvez indiquer des commentaires derrière des doubles tirets
```

Tout ce que vous écrivez après les doubles tirets ne sera pas exécuté, il est très utile de commenter votre code pour faciliter sa compréhension lors d'une relecture ultérieure ou par une autre personne.

Une fois la scène enregistrée via le bouton "Enregistrer", vous pouvez lancer votre scène à l'aide du bouton "Démarrer".

Ce script sera exécuté à chaque fois que vous cliquerez sur "Démarrer"

SCRIPT 2 (fonction "sleep")

Ce script permet d'allumer le module avec l'identifiant 2 et de l'éteindre après 3 secondes.

Nous allons utiliser la fonction "sleep" documentée ici (http://docs.fibaro.com/?page_id=1028).

Elle s'écrit comme ceci : fibaro:sleep(time) , l'argument "time" étant en milisecondes il faudra indiquer la valeur 1000 (ms) --> fibaro:sleep(1000)

Le scripte ressemblera donc à ceci :

```
1 --[[
2 %% properties
3 %% globals
4 --]]
5
6 fibaro:call(2, 'turnOn' ) -- allume la lampe
7 fibaro:sleep(3000) -- attend 3 secondes
8 fibaro:call(2, 'turnOff' ) -- éteint la lampe
```

SCRIPT 3 (fonction "debug")

Une autre fonction très utile est fibaro:debug. Cette fonction est un outil de déboguage très utile, il permet au développeur de contrôler exactement où le programme en est de son execution et permettre un deboguage rapide. Veuillez noter que dans Lua (comme dans tous les langages de programmation) le code du programme est exécuté ligne par ligne.

Description détaillée (http://docs.fibaro.com/?page_id=965) de la fontion "Debug"
(http://docs.fibaro.com/?page_id=965)

Elle s'écrit comme ceci : fibaro:debug(text)

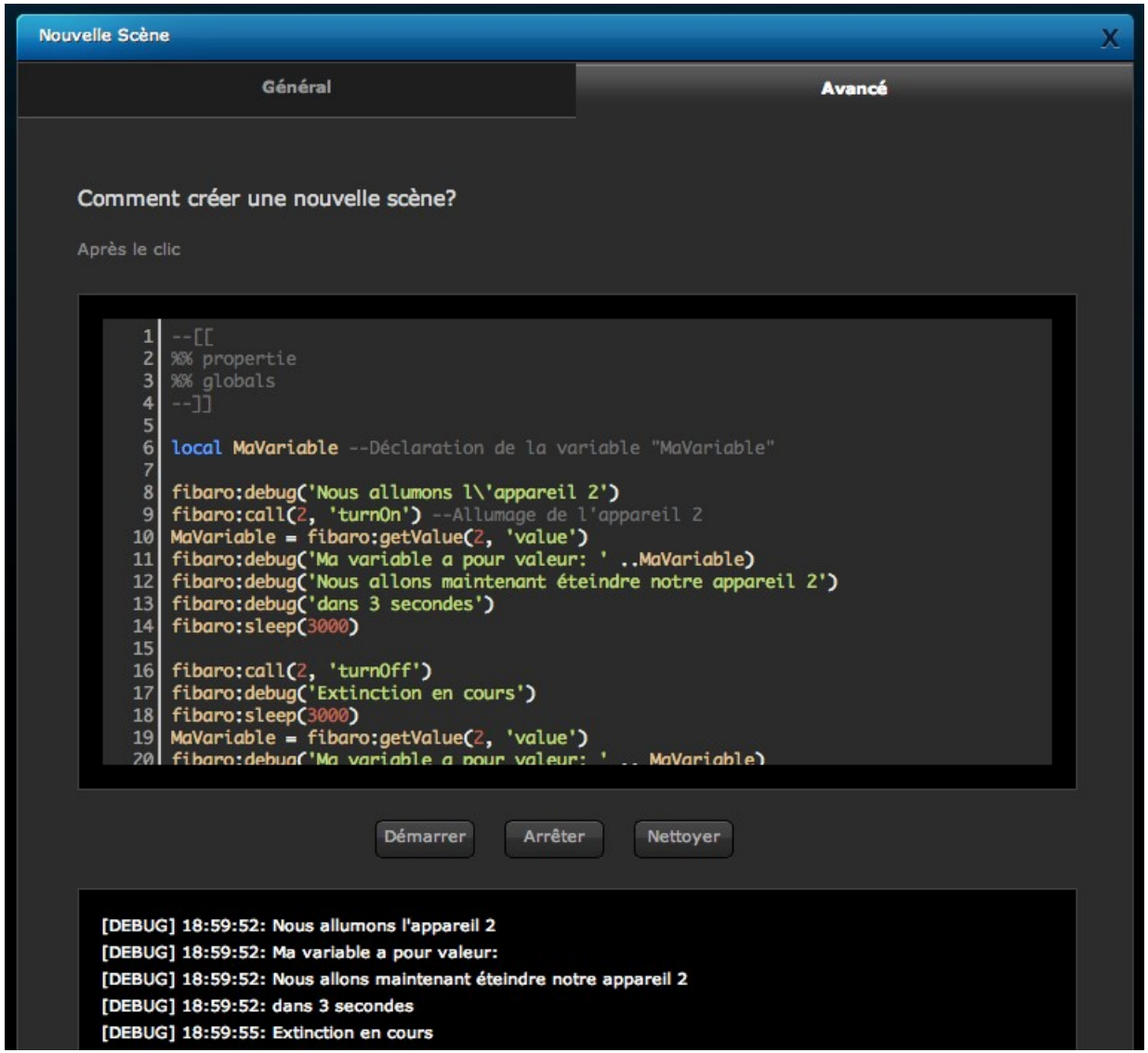
Lorsque vous exécuterez votre script, les informations de déboguage s'inscrirons sur l'écran noir en dessous du script.

Exemple :

```
--[[
%% properties
%% globals
  --]]

fibaro:debug( 'Hello world!' )
```

Va afficher ‘Hello world’ sur l'écran de déboguage.



SCRIPT 4 (setValue)

Dans cet exemple, nous allons utiliser les 3 fonctions vues précédemment (call mais cette fois complétée par un "setValue", wait et debug).

Nous souhaitons lors de notre départ de la maison exécuter le scénario suivant : fermer les volets en deux étapes avec un éclairage pendant l'opération, puis l'extinction de cet éclairage puis la fermeture totale du volet.

Utilisation de la fonction déjà vue "fibaro:call" mais complété par un "setValue" à 50%, ce qui donne : fibaro:call(10, ‘setValue’, ’50’)

Liste des équipements de notre exemple :

- Relais commutateur ON/OFF (deviceID 2) - Eclairage complémentaire
- Variateur (deviceID 5) - Eclairage principal
- Volet roulant (deviceID 10)

```
--[[
%% properties
%% globals
--]]

fibaro:debug( 'Démarrage du programme' )
fibaro:debug( 'J\'allume le relais ID 2' )
fibaro:call(2, 'turnOn' ) -- Allume le relais avec deviceID 2
fibaro:debug( 'J\'attends 5 secondes' )
fibaro:sleep(5000) -- Pause de 5 secondes
fibaro:debug( 'Les 5 secondes sont passées, je vais baisser le volet à 50%' )
fibaro:call(10, 'setValue' , '50' ) -- On baisse le volet à 50%
fibaro:debug( 'Volet en mouvement' )
fibaro:debug( 'j\'attends 5 secondes' )
fibaro:sleep(5000) -- On attends 5 secondes
fibaro:call(5, 'setValue' , '75' ) -- On fixe le variateur à 75%
fibaro:debug( 'L\'éclairage est réglé' )
fibaro:debug( 'Nous allons éteindre l\'éclairage' )
fibaro:call(2, 'turnOff' ) -- On éteints l\'eclairage complémentaire
fibaro:call(5, 'turnOff' ) -- On éteints le variateur
fibaro:debug( 'Nous allons fermer les stores' )
fibaro:call(10, 'turnOff' ) -- On ferme les stores
fibaro:debug( 'Tous les appareils sont éteints' )
```

Attention, vous remarquerez le caractère "\" --> "**antislash**" ou "**barre oblique inversée**" dans certains commentaires.

En effet c'est ce qu'on appelle un caractére d'échapement nécessaire pour que le système n'interprète pas l'apostrophe comme la fin du texte.

Exemples :

fibaro:debug('**mon texte de déboguage sans apostrophe**') -- pas de problème ici

fibaro:debug('**ici j**'ai mis une apostrophe sans échapemment') -- le système ne comprendra pas tout ce qui suite la deuxième apostrophe (qui indique la fin du texte), c'est à dire la phrase "ai mis une apos ..." va faire boguer le système.

fibaro:debug('**ici j**\'ai mis une apostrophe avec échapemment') -- ici, pas de problème, la 2e apostrophe est ignorée et la fin du texte est signalée par la 3e apostrophe.

SCRIPT 5 (les variables)

Dans cet exemple nous allons voir comment récupérer l'état / valeur d'un module (allumé, éteint, pourcentage d'ouverture, etc.).

Nous allons nous prendre pour exemple un relais commutateur FGS211 avec pour deviceID 2.

Ce module possède de nombreux paramètres mais pour le moment nous allons nous intéresser uniquement à la proprité "value"

"value" renvoie une valeur qui indique l'état de l'apareil (ex: 0 = éteint, 1 = allumé)

Pour cela nous allons utiliser la fonction fibaro:getValue documentée ici (http://docs.fibaro.com/?page_id=1009).

exemple d'utilisation : fibaro:getValue(deviceID, propertyName).

Maintenant, nous allons passer à la prochaine **information très importante**:
Pour stocker une valeur dans n'importe quel langage de programmation vous utilisez ce qu'on appelle des **variables**. La variable est un concept, qui dans les langages de programmation (y compris LUA) a trois caractéristiques - le **type**, le **nom** et la **valeur**.

Voici la structure des variables en LUA (plus d'informations ici (http://docs.fibaro.com/?page_id=1067))

```
1 local MaVariable
```

Nous devons déclarer les variables locales.

Elles ne sont visibles que dans notre script.

Le variables sont un élément essentiel d'un langage de programmation, elles permettent de stocker des valeurs numériques ou des chaines de caractères (texte).

Il est possible de stocker dans une variable une valeur provenant de l'état d'un module.

```
1 local MaVariable = fibaro:getValue(2, 'value')
```

Récupère l'état (0 = éteint, 1= allumé) de notre module deviceID 2 et stock cette valeur dans la variable "MaVariable"

Exemple :

```
1 --[[
2 %% properties
3 %% globals
```

```
4 --]]
5
6 local MaVariable --Déclaration de la variable "MaVariable"
7
8 fibaro:debug( 'Nous allumons l\'appareil 2' )
9 fibaro:call(2, 'turnOn' ) --Allumage de l'appareil 2
10 MaVariable = fibaro:getValue(2, 'value' )
11 fibaro:debug( 'Ma variable a pour valeur: ' ..MaVariable)
12 fibaro:debug( 'Nous allons maintenant éteindre notre appareil 2' )
13 fibaro:debug( 'dans 3 secondes' )
14 fibaro:sleep(3000)
15
16 fibaro:call(2, 'turnOff' )
17 fibaro:debug( 'Extinction en cours' )
18 fibaro:sleep(3000)
19 MaVariable = fibaro:getValue(2, 'value' )
20 fibaro:debug( 'Ma variable a pour valeur: ' .. MaVariable)
21
22 fibaro:debug( 'Nous allumons de nouveau notre module' )
23 fibaro:call(2, 'turnOn' )
24 fibaro:sleep(3000)
25 MaVariable = fibaro:getValue(2, 'value' )
26 fibaro:debug( 'Ma variable a pour valeur: ' .. MaVariable)
```

Premièrement nous déclarons la variable (ligne 6),

nous allumons l'appareil 2 (ligne 9),

nous stockons l'état de l'appareil récupéré par la fonction "getValue" dans la variable précédemment créé (ligne 10),

nous affichons le contenu de la variable dans le débogue en ajoutant ..MaVariable après le texte délimité entre les apostrophes (ligne 11).

`fibaro:debug( 'Ma variable a pour valeur: ' ..MaVariable)`

donnera sur l'écran de débogue :

[DEBUG] 17:57:55: Ma variable a pour valeur: 1

Il aurait aussi été possible de regrouper les étapes des lignes 6 et 10 en tapant :

local `MaVariable = fibaro:getValue(2, 'value' )`

SCRIPT 6 (fonction IF)

Maintenant, nous introduisons un autre élément de langages de programmation qui est IF. Une instruction conditionnelle permet d'exécuter du code de programme, sous certaines conditions, qui est dans l'exemple ci-après, SI (IF) l'appareil deviceID 2 est allumé ALORS (THEN) allumer le deviceID 3.

```
1 if (état) then
2
3 instruction
4
5 end
```

Ce qui donne :

```
1 --[[
2 %% properties
3 %% globals
4 --]]
5 local MaVariable --déclaration de la variable "MaVariable"
6
7 fibaro:call(2, 'turnOn' ) --j'allume l'appareil 2
8 MaVariable = fibaro:getValue(2, 'value' )
9
10 if (MaVariable == '1' ) then
11 fibaro:call(3, 'turnOn' )
12 end
```

On déclare la variable (ligne 5),

on allume l'appareil 2 (ligne 7),

on récupère l'état de l'appareil 2 et le stockons dans "MaVariable" (ligne 8),

si (if) "MaVariable" est égale (==) à "1" alors (then) (ligne 10) allumer l'appareil 3 (ligne 11) fin de la condition IF (end) (ligne 12).

Si la valeur de MaVariable était à "0" alors le programme serait allé directement après le "end" donc dans le cas présent à la fin du programme (sans allumer l'appareil 3).

SCRIPT 7 (fonction ELSE)

Nous avons un détecteur d'ouverture dont le deviceID est 145, il possède de nombreuses propriétés comme par exemple le niveau de charge de sa pile, s'il est armé, etc ...).

Nous n'allons pour le moment nous intéresser qu'à la propriété "Value" (0 = fermé / intact , 1 = ouvert / violé).

Si nous voulons vérifier l'état du capteur nous pouvons utiliser la commande suivante :

```
1 fibaro:getValue(145, 'value' )
```

Si nous voulions savoir si le capteur est armé, nous aurions utilisé la commande suivante:

```
1 fibaro:getValue(145, 'armed' )
```

Dans les deux cas, le résultat est une valeur binaire 0 ou 1.

```
--[[
%% properties
%% globals
--]]

local MaVariable1 = fibaro:getValue(145, 'value') --stock la valeur du capteur 145 dans MaVariable1
fibaro:debug( 'value = ' .. MaVariable1) --Affiche la valeur de MaVariable1
local MaVariable2 = fibaro:getValue(2, 'value') --stock la valeur de l'interrupteur 2 dans MaVariable2
fibaro:debug( 'value = ' .. MaVariable2) --Affiche la valeur de MaVariable2

if (MaVariable1 == '1') then

    if (MaVariable2 == '0') then
        fibaro:call(2, 'turnOn')
    else
        fibaro:call(2, 'turnOff')
    end

end
```

autrement dit (lignes 11 à 19):

SI le capteur de porte est ouvert

ALORS :

SI la lampe est éteint **ALORS** l'allumer

SINON éteindre la lampe

FIN (du SI n°2)

FIN (du SI N°1)

SCRIPT 8 (les trigger)

Pour le moment nous n'avons montré que des script qui nécessitaient d'être executés manuellement mais il pourrait être intéressent de les déclencher automatiqument en fonction d'un événement.

C'est là qu'entre en scène les **trigger** (déclencheurs).

Exemple: nous voudrions que la lumière s'allume automatiquement lorsque le capteur détecte un mouvement.

```
local MaVariable --je déclare MaVariable
```

```
2
3 MaVariable = fibaro:getValue(50, 'value')
4 if (MaVariable == '1') then
5     fibaro:call(3, 'turnOn')
6 end
```

Ce qui signifie : SI le capteur de mouvement 50 est en alarme (mouvement détecté) ALORS allumer la lampe 3.

Le problème c'est que ce script devrait s'exécuter en permanence (en faisant des boucles) pour fonctionner mais ce n'est pas une bonne solution.

Nous voudrions que ce script s'execute lorsque le capteur passe en détection de mouvement, pour cela nous allons utiliser le déclancheur "TRIGGER"

Pour cela nous allons devoir déclarer dans l'entête le DeviceID du capteur (50), suivi de la propriété servant de déclencheur (value):

```
1 --[[
2 %% properties
3 50 value
4 %% globals
5 -]]
```

Maintenant, à chaque fois que la propriété "valeur" du capteur changera (détection ou fin de détection), le programme s'exécutera.

La condition SI servira quand à elle à vérifier s'il s'agit bien d'une détection de mouvement (value == 1) et pas une fin de détection (value == 0)

```
1 %% properties
2 50 value
3 %% globals
4 -]]
5
6 local MaVariable --déclaration de MaVariable
7
8 fibaro:debug('démarrage du programme')
9 MaVariable = fibaro:getValue(50, 'value')
10 if (MaVariable == '1') then
11     fibaro:call(3, 'turnOn')
12 end
```

Si nous voulons aussi éteindre la lumière lorsque que le capteur passe en "fin de détection" (value == 0), il suffit d'ajouter quelques lignes :

```
1 --[[
2 %% properties
3 50 value
4 %% globals
5 --]]
```

```
6
7  local MaVariable --déclaration de MaVariable
8
9  MaVariable = fibaro:getValue(50, 'value')
10 if (MaVariable == '1') then
11     fibaro:call(3, 'turnOn')
12 else
13     fibaro:call(3, 'turnOff')
14 end
```

SCRIPT 9 (os.date)

Ce script est conçu pour allumer deux lampes dans un intervalle de 5 secondes l'une de l'autre après ouvrture de la porte; de plus cette scène doit fonctionner uniquement entre 15h et 18h.

```
1  --[[
2  %% properties
3  <CapteurPorte> value
4  %% globals
5  --]]
6
7  local Maintenant = os.date("*t")
8
9  if (Maintenant['hour'] >= 15 and Maintenant['hour'] <= 18) then
10     fibaro:call(<lampe1>, 'turnOn')
11     fibaro:sleep(5000)
12     fibaro:call(<lampe2>, 'turnOn')
13 end
```

Pour connaitre les autres fonctions liées aux dates et heures vous pouvez consulter la documentation sur LUA.org (http://www.lua.org/pil/22.1.html)

```
1 local Maintenant = os.date("*t") --stocke la date et heure actuelle du système sous forme d'un tableau à l'intérieur de la variable "Maintenant"
```

Il est possible de récupérer les champs suivants stockés dans ce tableau :

`year`	Année
`month`	Mois de 01 à 12
`day`	Jours de 01 à 31
`hour`	Heures de 00 à 23

`min`	Minutes de 00 à 59
`sec`	Secondes de 00 à 59
`isdst`	est binaire, renvoie "true" si nous sommes en heure d'été

Pour comparer l'heure actuelle avec nos horaires de consigne on utilise la ligne suivante avec un "AND" (et):

```
1 if( Maintenant[ 'hour' ] >= 15 and Maintenant[ 'hour' ] < 18) then
```

La condition est : si il est (Maintenant) "15h ou plus" et "moins de 18h" --> donc entre 15h et 17h59

Dans le cas d'une plage horaire se situant sur deux dates, typiquement entre avant minuit et après minuit, nous devons utiliser la ligne suivante avec un "OR" (ou):

```
1 if( Maintenant[ 'hour' ] >= 23 or Maintenant[ 'hour' ] < 4)
```

La condition est : si il est (maintenant) "23h ou plus" ou "moins de 4h" --> donc entre 23h et 3h59.

Pour compléter ce tutoriel :

- Un excellent et très complet article de nos confrères de **domotique-fibaro.fr** :
  - Introduction (http://www.domotique-fibaro.fr/index.php/cp/2-api-fibarofr/)
  - Généralités (http://www.domotique-fibaro.fr/index.php?app=cp&do=show&pageId=5)
  - Index des fonctions (http://www.domotique-fibaro.fr/index.php?app=cp&do=show&pageId=3)
- Utilisation des télécommandes (ZME et Aeon) comme déclencheur de script LUA (http://tutoriels.domotique-store.fr/content/5/81/fr/lua-sur-hc2-declencher-des-scenes-avec-les-telecommandes-z_waveme-kfob-aeon-dsa22-keyfob-dsa38-panic-ou-dsa03-minimote.html)
- Documentation des fonctions (http://docs.fibaro.com/?page_id=870) (Officiel / Polonais)
- Résumé des fonctions par thématiques (http://docs.fibaro.com/?page_id=1129) (Officiel / Polonais)
- Documentation des appareils (http://docs.fibaro.com/?page_id=1090) (Officiel / Polonais)
- Bases du LUA : conditions, variables et opérateurs (http://docs.fibaro.com/?page_id=1067) (Officiel / Polonais)
- Manuel officiel du LUA (http://www.lua.org/manual/5.1/) (Anglais)

Quelques commandes incontournables pour appeller des API externes:

- Net.FUdpSocket (http://docs.fibaro.com/?page_id=1900) (Anglais)
- Net.FTcpSocket (http://docs.fibaro.com/?page_id=1921) (Anglais)
- Net.FHttp (http://docs.fibaro.com/?page_id=1930) (Anglais)
- json (http://docs.fibaro.com/?page_id=2008) (Anglais)

Recommandez cet article :

+2 Recommander ce contenu sur Google
Tweeter
J'aime 0 Recommander Partager 0

Ces articles pourraient aussi vous intéresser :

- Historique des mises à jour du logiciel du Fibaro Home Center (/content/76/34/fr/historique-des-mises-a-jour-du-logiciel-du-fibaro-home-center.html)
- Gérer son arrosage en domotique avec le Home Center (/content/76/159/fr/gerer-son-arrosage-en-domotique-avec-le-home-center.html)
- Mise en service du Home Center 2 (/content/76/70/fr/mise-en-service-du-home-center-2.html)
- Mise en service du Home Center Lite (/content/77/156/fr/mise-en-service-du-home-center-lite.html)
- Manuel du Home Center 2 (/content/74/161/fr/manuel-du-home-center-2.html)

A propos de cet article :

Dernière mise à jour :
2014-05-13 15:49
Révision:
1.1
Auteur :
Jérôme Massiaux, technicien domotique-store.fr (http://www.domotique-store.fr)

0 Commentaires Tutoriels et Manuels www.domotique-store.fr S'identifier

Recommend Partager Les plus récents

Start the discussion…

Be the first to comment.

AUSSI SUR TUTORIELS ET MANUELS WWW.DOMOTIQUE-STORE.FR

Questions et avis sur QUBINO ZMNHJD1 - Micromodule "fil pilote V2" Z-Wave+ (remplace le …

74 commentaires • il y a 6 mois

Sylvain - Domotique-Store.fr — Bonjour Sébastien, Le module Qubino ZMNHJD1 n'agit pas comme un relais de puissance. Ce module fil pilote ne supporte pas …

Questions et avis sur Philio "PSR04 Smart Color Button" Télécommande murale Z-Wave+

2 commentaires • il y a 5 mois

Boris - Domotique-Store — Bonjour, Il est vrai que le Bip ne peut être désactivé, ce qui peut être très gênant et très peu discret. Sincèrement désolé que cet interrupteur ne vous …

Questions et avis sur FOSCAM FI9900P - Caméra IP Wifi HD 1080P pour utilisation extérieure

2 commentaires • il y a 5 mois

Damien - Domotique-Store.fr — Bonjour, Je vous confirme que cette caméra ne permet de régler que l'inclinaison verticale. Bien cordialement, Damien Service Clients …

Questions et avis sur DANFOSS Sonde d'ambiance Z-Wave Link RS 014G0160

10 commentaires • il y a 5 mois

Damien - Domotique-Store.fr — Bonjour, Effectivement il est possible de contacter Fibaro afin de leur demander d'ajouter la compatibilité avec ce module. Cependant, les …

S'abonner Ajoutez Disqus à votre site web ! Add Disqus Add Vie Privée

© 2013-2015 - Tous droits réservés - www.domotique-store.fr (http://www.domotique-store.fr) - Informations non contractuelles
Sous réserve d'exactitude des informations fournies par le constructeur et d'erreur typographique ou de traduction.
En cas de doute, veuillez consulter la version originale et officielle du manuel utilisateur.
En aucun cas la responsabilité de SIRPEA SARL / domotique-store.fr où même du rédacteur de l'article ne pourra être engagée en cas de dommage matériel ou immatériel.
Il incombe au lecteur de prendre ses précautions en terme de sécurité électrique notament.